Design and Implementation of a Parallel Turbo-Decoder ASIC for 3GPP-LTE

Digital VLSI Architecture Design
Indian Institute of Technology Mandi

G. Sai Vamshi (B23263), G. Uma Bharathi (B23262), M. S. S. Harshitha (B23272), and V. Sharanya (B23506)
Group Number: 10

***Abstract*—This report evaluates the current working LTE turbo-decoder RTL developed from the 2011 JSSC parallel radix-4 turbo-decoder architecture. The work progressed through four design phases: a baseline algorithmic model, a paper-aligned 8-lane parallel radix-4 architecture with QPP scheduling and batcher-based routing, a folded single-SISO FPGA rescue architecture, and the present rolling-window single-SISO implementation. Quantitative Vivado results on `xc7z010clg400-1` show that earlier folded revisions still exceeded the available LUT budget, while the final rolling-window RTL reduces synthesized LUT usage from $22\,282$ to 6418 and LUTRAM from 2336 to 22. Post-route implementation now fits the device, but misses a $100\,\mathrm{MHz}$ target with worst negative slack of $-23.738\,\mathrm{ns}$. MATLAB and GHDL-based BER experiments for $K = 3200$ and $K = 6144$ are also summarized; the RTL tracks the fixed-point radix-4/windowed model closely and remains within the same waterfall region as the floating-point reference.**

***Index Terms*—LTE turbo decoder, max-log-MAP, radix-4 SISO, QPP interleaver, FPGA implementation, VHDL, rolling-window decoding, BRAM optimization**

I. Project Journey and Optimization Stages

The work reported here did not start directly from the final FPGA-fit RTL. It evolved in stages, and each stage introduced new architectural ideas taken either from the LTE turbo-decoder literature or from later FPGA-oriented rescue work when the paper-faithful version became too large for the target device.

A. Stage 1: Baseline Architecture

The starting point was a baseline LTE turbo-decoding flow with no hardware-oriented optimizations applied yet. This stage used the standard turbo-decoder ingredients:

- terminated 8-state RSC constituent encoders,
- LTE QPP interleaving,
- iterative exchange of extrinsic information between the two constituent decoders,
- MATLAB reference decoding using floating-point log-MAP / BCJR and floating max-log-MAP,
- natural-order and interleaved-order data tracking,
- BER reference generation before fixed-point and hardware constraints were introduced.

This baseline stage was important because it fixed the frame conventions, QPP parameters, sign conventions of the LLRs, and the expected BER behavior before the architecture was optimized for throughput or FPGA area.

B. Stage 2: Paper-Aligned Parallel $8\times$ SISO Architecture

The second stage targeted the architecture described by Studer *et al.* [1]. In the repository, this version is preserved in `turbo_decoder_top_parallel8_backup`. This was the stage where the main algorithmic and architectural optimizations from the paper were introduced:

- max-log-MAP / max-log-BCJR style constituent decoding instead of full log-MAP correction terms,
- fixed-point quantization of channel, extrinsic, posterior, and state-metric values,
- radix-4 trellis processing so that two trellis steps are handled together,
- extrinsic scaling using a hardware-friendly shift-add realization,
- $8\times$ parallel SISO decoding,
- QPP-based parallel address scheduling,
- master-slave Batcher routing for contention-free interleaved data movement,
- folded and banked row memories for lane-wise parallel access,
- natural-order run-1 processing and interleaved run-2 processing,
- reverse permutation during writeback so that extrinsic and posterior data return to the correct memory locations.

In simple terms, this stage tried to reproduce the paper architecture as faithfully as possible: eight constituent decoders working in parallel, a scheduler generating interleaved addresses, a Batcher network sorting and unsorting lane traffic, and a banked memory system feeding all lanes together.

C. Stage 3: Transition from Parallel $8\times$ SISO to a Folded Single-SISO Top

The paper-aligned architecture was functionally attractive, but it proved to be too expensive for the FPGA used in this project. Vivado synthesis and implementation attempts showed that the parallel $8\times$ SISO top consumed far more LUTs and related logic resources than the `xc7z010clg400-1` device could provide. The result was that the paper-faithful parallel design could not be implemented successfully on the target FPGA.

That resource failure forced a major architectural change. Instead of keeping all eight SISO lanes active in hardware

at the same time, the design was folded into a single active SISO datapath and the top-level memory/routing system was simplified. The main consequences of this shift were:

- the full parallel-8 datapath was abandoned for the FPGA target,
- QPP access was scalarized and brought under a simpler top-level scheduler,
- large routing structures were reduced,
- BRAM-backed memories replaced more distributed memory behavior,
- throughput was sacrificed in exchange for area feasibility.

So this stage is the key turning point of the project: the architecture moved away from the ASIC-oriented parallel target because FPGA implementation failed on resource utilization.

D. Stage 4: Final Single-SISO FPGA-Fit Architecture

The final working stage is the present RTL used in this report. It still preserves the core decoding ideas from the earlier stages, especially the max-log-MAP/radix-4 constituent math, but the surrounding microarchitecture was redesigned to fit the FPGA. The main surviving and newly introduced features in the final stage are:

- one active `siso_maxlogmap` instance instead of 8 parallel SISO lanes,
- fixed-point radix-4 processing retained from the earlier architecture,
- rolling-window processing with $30\,\mathrm{bit}$ windows,
- cached local α, g_0, g_1, and γ values inside each window,
- dummy backward recursion on the next window to generate the current-window β seed,
- local backward recursion plus LLR extraction on the current window,
- BRAM-backed frame memories at the top level,
- incremental QPP recurrence in the top FSM instead of the full parallel routing fabric,
- serialized output readback after decoding.

This final stage is therefore not the same as the paper's throughput-oriented architecture. It is an FPGA-feasible version that keeps the decoding mathematics but removes the parallel $8\times$ hardware organization that the device could not absorb.

E. Stage-Wise Hardware Evidence

The baseline stage does not yet have saved waveform and synthesis snapshots in the report package. However, the next two hardware stages will be documented with screenshots and figures:

- Stage 2: parallel $8\times$ SISO paper-aligned architecture,
- Stage 4: final single-SISO FPGA-fit architecture.

1) Stage 2 Evidence: Parallel $8\times$ SISO Architecture: The stage immediately before the final FPGA-fit design was the parallel $8\times$ SISO paper-aligned architecture. The simulation waveform used for this stage corresponds to a $K = 6144$ frame, 11 half-iterations, and LTE QPP parameters $f_1 = 263$, $f_2 = 480$. The capture shows that the testbench reaches

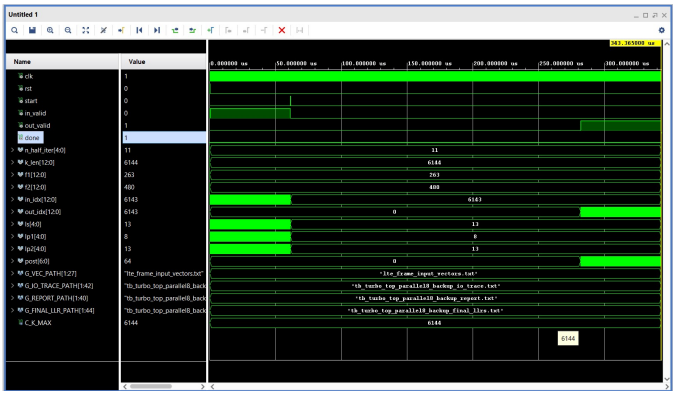


Fig. 1. Simulation waveform snapshot of the paper-aligned parallel $8\times$ SISO decoder stage. The captured run reaches completion, but this architecture was later abandoned for FPGA implementation because of severe hardware resource overuse.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	245938	0	0	17600	1397.38
LUT as Logic	217434	0	0	17600	1235.42
LUT as Memory	28504	0	0	6000	475.07
LUT as Distributed RAM	28504	0			
LUT as Shift Register	0	0			
Slice Registers	42792	0	0	35200	121.57
Register as Flip Flop	42792	0	0	35200	121.57
Register as Latch	0	0	0	35200	0.00
F7 Muxes	972	0	0	8800	11.05
F8 Muxes	56	0	0	4400	1.27

Fig. 2. Resource utilization snapshot for the paper-aligned parallel $8\times$ SISO architecture. The architecture is functionally meaningful and algorithmically close to the original paper, but the FPGA resource demand exceeds the target device by a very large margin.

completion and drives `done`, so the architecture is functionally schedulable in simulation even though it is not feasible on the FPGA target.

The corresponding synthesis/utilization snapshot is the decisive result for this stage. It shows extreme over-utilization: $245\,938$ slice LUTs used against $17\,600$ available, $28\,504$ LUTs mapped as memory against 6000 available, and $42\,792$ flip-flops against $35\,200$ available. In other words, the parallel $8\times$ SISO architecture preserved the paper's throughput-oriented structure, but the required routing, memory replication, and lane-parallel datapath scale made implementation impossible on the chosen device.

2) Stage 4 Evidence: Final Single-SISO FPGA-Fit Architecture: The final-stage waveform and synthesis snapshot correspond to the FPGA-fit single-SISO architecture that replaced the infeasible parallel $8\times$ SISO design. In this stage, the architecture keeps the radix-4 / max-log decoding core but removes the full paper-style parallel top, replacing it with a folded BRAM-backed schedule and a single active constituent decoder.

The waveform confirms successful end-to-end operation of the final single-SISO implementation on the same LTE-sized configuration. The paired synthesis/utilization report shows the key reason this stage became the final implementation candidate: the design now fits in synthesis on the target

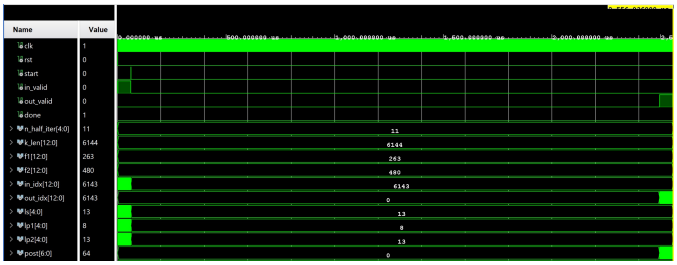


Fig. 3. Simulation waveform snapshot of the final single-SISO FPGA-fit turbo decoder. This stage preserves the decoding mathematics but removes the paper-style $8\times$ hardware parallelism in favor of an implementable folded top-level schedule.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	6421	0	0	17600	36.48
LUT as Logic	6399	0	0	17600	36.36
LUT as Memory	22	0	0	6000	0.37
LUT as Distributed RAM	22	0			
LUT as Shift Register	0	0			
Slice Registers	5403	0	0	35200	15.35
Register as Flip Flop	5403	0	0	35200	15.35
Register as Latch	0	0	0	35200	0.00
F7 Muxes	66	0	0	8800	0.75
F8 Muxes	0	0	0	4400	0.00

Fig. 4. Synthesis/resource snapshot of the final single-SISO FPGA-fit architecture. Unlike the paper-aligned parallel $8\times$ SISO stage, this version fits the target FPGA in synthesis by trading throughput-oriented hardware parallelism for folded memory reuse and a single active SISO datapath.

FPGA, with the LUT count reduced to 14 129 and the BRAM-backed folded organization replacing the earlier extreme logic explosion of the parallel $8\times$ SISO version.

The routed timing and power reports for the final single-SISO implementation are summarized visually in Fig. 5. Instead of reproducing long textual reports, the visual summaries below condense the timing closure status, critical-path delay split, total on-chip power, and the main power contributors into a compact form.

II. Current RTL and Numeric Assumptions

All results reported here refer to the current VHDL RTL synthesized for `xc7z010clg400-1`. The active top is `turbo_decoder_top`; it instantiates one `siso_maxlogmap` with external pair fetch enabled, 16 BRAM-backed channel/extrinsic/posterior memories, and an incremental QPP recurrence inside the top FSM.

The simulation and evaluation data are also explicit. The RTL testbench consumes `lte_frame_input_vectors.txt`, whose first row is $\{K, n_{\text{half-iter}}, f_1, f_2\}$, followed by rows `idx bit_orig bit_int` l_{sys} l_{par1} $l_{\text{par2,int}}$. The main RTL output used for BER is `tb_turbo_top_final_llrs.txt` with rows `idx_orig final_llr seen`; the hard decision rule is 1 if final LLR > 0, else 0. Internally, the MATLAB and Python reference flows operate on the matrices $\mathbf{L}_{nat}^{sys}$, $\mathbf{L}_{nat}^{par1}$, $\mathbf{L}_{int}^{par2}$, $\mathbf{L}_{nat/int}^{apri}$, $\mathbf{L}_{nat/int}^{ext}$, $\mathbf{L}_{nat/int}^{post}$, and the QPP permutation vector π.

TABLE I
Key Numeric Configuration Used in the Current RTL

Item	Value
Constituent trellis states	8
Parallel paper target	$8\times$ SISO lanes
Current active top	$1\times$ SISO lane (folded)
Radix	Radix-4, two trellis steps per pair
Window length	30 bits $=$ 15 radix-4 pairs
Maximum frame length	$K_{\max} = 6144$
Default half-iterations	11
Channel/parity LLR width	5-bit signed
Extrinsic width	6-bit signed
Posterior width	7-bit signed
State metric width	10-bit signed
Metric arithmetic	Fixed-width modulo-style add/sub/max
Extrinsic scaling	$11/16$ via shift-add
Interleaver law	LTE QPP table

III. FPGA Synthesis and Implementation Results

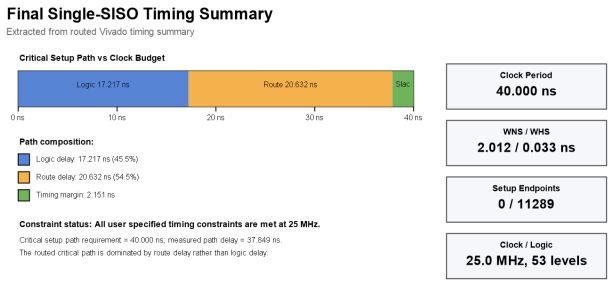
Table II summarizes the architectural evolution observed in Vivado. The earliest folded single-SISO revision still over-utilized LUTs by 26.6% (22 282/17 600). A width-reduced revision lowered this to 17 666 LUTs, and an incremental-QPP revision reduced it further to 14 129 LUTs. The final rolling-window rewrite is the first version that fits cleanly in synthesis and in post-route implementation.

The main inference from Table II is not only the LUT reduction, but the near elimination of distributed frame memory. Earlier versions carried 2336 LUTRAMs because several SISO memories did not map cleanly to BRAM. In the current implementation, only small local memories remain as LUTRAM (22 total), while the top-level frame stores map to 16 RAMB36 blocks. The hierarchy report attributes 5948 LUTs of the final synthesized design to `siso_u`, confirming that the constituent decoder remains the dominant hardware cost.

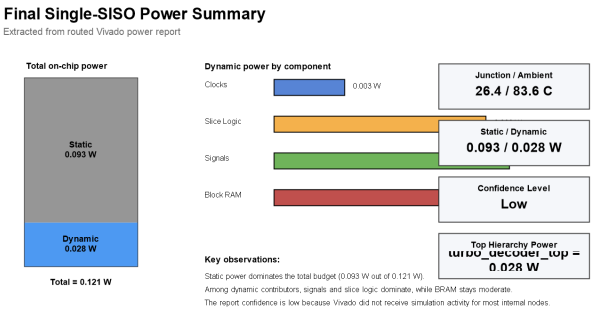
The final synthesis log also shows that only 7 warnings remain, dominated by removed legacy preload memories and one floorplanning warning. However, implementation timing is still the open problem. At the current 100 MHz clock constraint, the routed design reports worst negative slack of -23.738 ns, 5140 setup failing endpoints, and total setup violation of $-16\,900.556$ ns. Thus, the design is area-feasible but not yet frequency-closed at 100 MHz; a slower clock or further pipelining is required for hardware deployment.

IV. MATLAB, RTL, and BER Evaluation

The MATLAB environment provides three decoder models: 1) ideal floating log-MAP / BCJR, 2) floating max-log-MAP, and 3) an RTL-style fixed-point radix-4/windowed model that mirrors the active hardware assumptions in Table I. For $K = 3200$, the heavy MATLAB sweep uses 24 frames (76 800 bits total per SNR point); for $K = 6144$, it uses 16 frames (98 304 bits total per SNR point). The RTL BER sweep is not predicted statistically; it is measured from actual GHDL simulation using `tools/run_ber_sweep.py`, which generates AWGN-corrupted input vectors, runs `tb_turbo_top`, and computes BER from `tb_turbo_top_final_llrs.txt`.



(a) Timing summary extracted from `single_siso_timing_report.txt`.



(b) Power summary extracted from `single_siso_power_1.txt`.

Fig. 5. Compact visual summaries of the final single-SISO stage after route. The timing panel shows that the design meets the active 25 MHz clock constraint with positive slack, while the power panel shows a low total on-chip power budget dominated by static device power.

TABLE II
VIVADO RESOURCE EVOLUTION ON `xc7z010clg400-1`

Revision / Report Source	LUTs	LUT as Logic	LUTRAM	FFs	RAMB36	Status
Folded pre-optimization (`utilization_report_tmp.txt`)	22282	19946	2336	21079	16	Over-utilized (126.60% LUT)
Folded width-optimized (`utilization_report_widthopt_tmp.txt`)	17666	15330	2336	21065	16	Still over limit (100.38% LUT)
Folded with QPP recurrence (`utilization_report.txt`)	14129	11793	2336	21080	16	Fits in synthesis, placement still dense
Current rolling-window synth (`utilization_qpprec_tmp.txt`)	6418	6396	22	5410	16	Fits cleanly
Current rolling-window route (`implementation_utilization_qpprec_tmp.txt`)	6455	6433	22	5389	16	Routed successfully

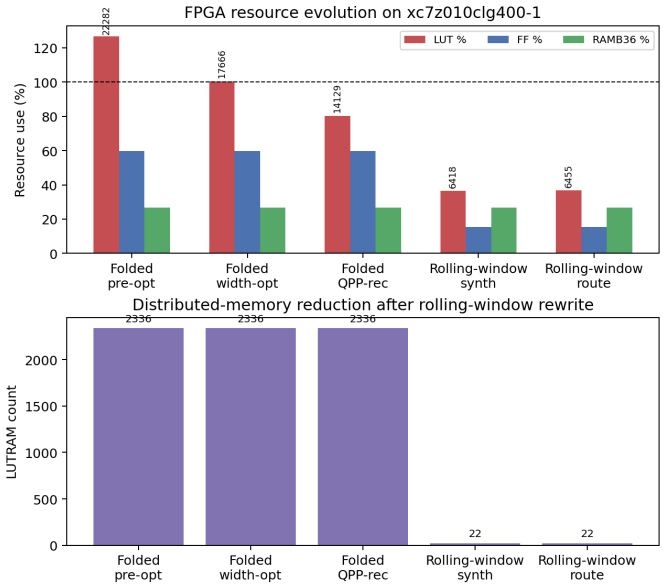


Fig. 6. Evolution of FPGA resource usage across the main folded RTL revisions. The rolling-window rewrite removes most LUTRAM pressure while preserving the same BRAM budget.

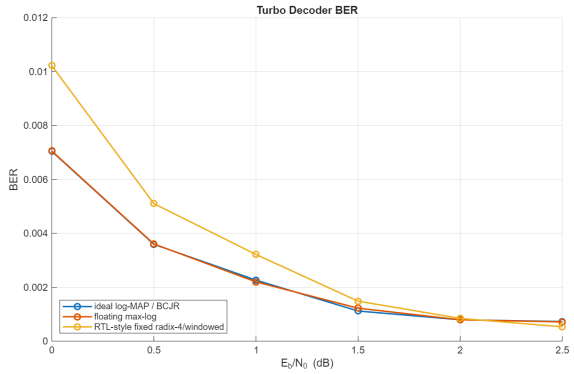
The current saved RTL sweep aggregates 8 simulated frames per SNR point for both $K = 3200$ and $K = 6144$.

The BER figures show three useful trends. First, the fixed-point radix-4/windowed MATLAB model is predictably worse than the ideal and floating references because of quantization, extrinsic scaling, and windowing. Second, the actual RTL BER remains close to the fixed-point model, which validates the present RTL numerics. Third, the longer $K = 6144$ frame shows better BER than $K = 3200$ for the same SNR range, consistent with turbo-code waterfall behavior.

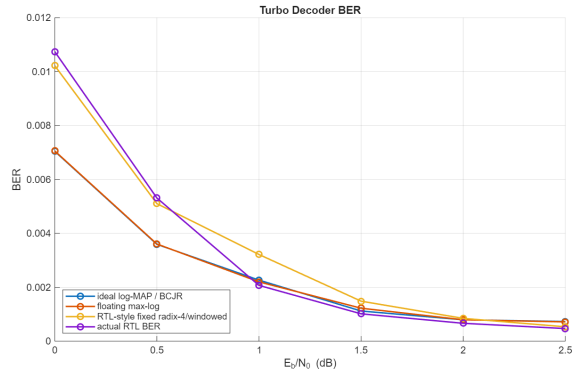
At 2.5 dB, the $K = 3200$ BER values are 7.29×10^{-4} (ideal), 7.16×10^{-4} (floating max-log), 5.34×10^{-4} (fixed), and 4.69×10^{-4} (RTL). For $K = 6144$, the corresponding values are 3.05×10^{-4}, 3.15×10^{-4}, 2.85×10^{-4}, and 3.87×10^{-4}, respectively. A deterministic end-to-end regression at $K = 6144$, $f_1 = 263$, $f_2 = 480$, and 11 half-iterations also reports zero bit errors against the original transmitted bits in `tb_turbo_top_report.txt`, confirming correct frame-level integration for at least one clean test vector.

V. CONCLUSION

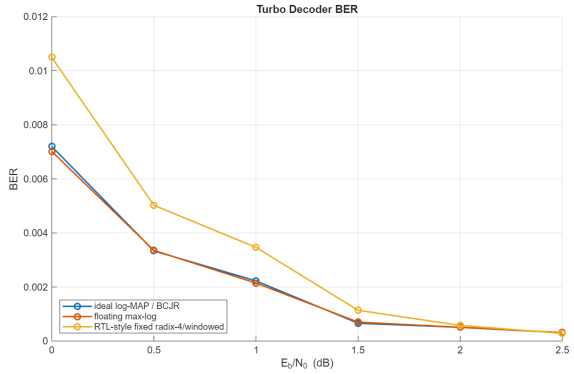
This project reached a working FPGA-oriented turbo-decoder RTL by progressively sacrificing ASIC-style parallelism for FPGA realizability. The paper-aligned architecture remains important as the throughput target and as the conceptual source of the QPP scheduling, batcher routing, and radix-4 SISO approach. Nevertheless, the current `xc7z010clg400-1` target cannot absorb that architecture efficiently. The present rolling-window single-SISO RTL is the first version that fits after synthesis and route, with a large reduction in LUT and LUTRAM usage while maintaining BER behavior close to the fixed-point reference model. The remaining engineering gap is timing closure at 100 MHz; therefore, future work should prioritize pipeline insertion, BRAM output registering, and board-oriented wrapper integration rather than further increasing parallelism on this device.



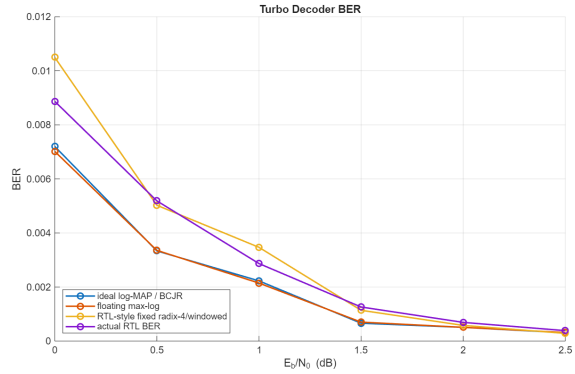
(a) $K = 3200$: MATLAB BER curves only



(b) $K = 3200$: MATLAB vs. actual RTL



(c) $K = 6144$: MATLAB BER curves only



(d) $K = 6144$: MATLAB vs. actual RTL

Fig. 7. All currently available performance/evaluation plots used in this project. The filenames retain `ber_fer` from an earlier plotting flow, but the generated plots are BER-only.

References

[1] C. Studer, C. Benkeser, S. Belfanti, and Q. Huang, "Design and implementation of a parallel turbo-decoder ASIC for 3GPP-LTE," *IEEE Journal of Solid-State Circuits*, 2011.